VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Aim: To Simulate the following CPU scheduling algorithms a) FCFS b) SJF

(i)FCFS without Arrival time

Algorithm:

1.Start
2.Read n
3.Read burst time (BT) of all processes
4.Set WT[0]=0
5.For i=1 to n-1
WT[i]=WT[i-1]+BT[i-1]
6.For i=0 to n-1
TAT[i]=WT[i]+BT[i]
7.Calculate average waiting time
8.Calculate average turnaround time
9.Display BT,WT,TAT,average WT and average TAT
10.Stop

Problem: Consider the set of processes that arrive at time 0 with the burst time given in milliseconds.

Process	Burst Time (ms)
P1	24
P2	3
P3	3

Solution:

Program: Source code

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst Time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for(i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

VNR VJIET

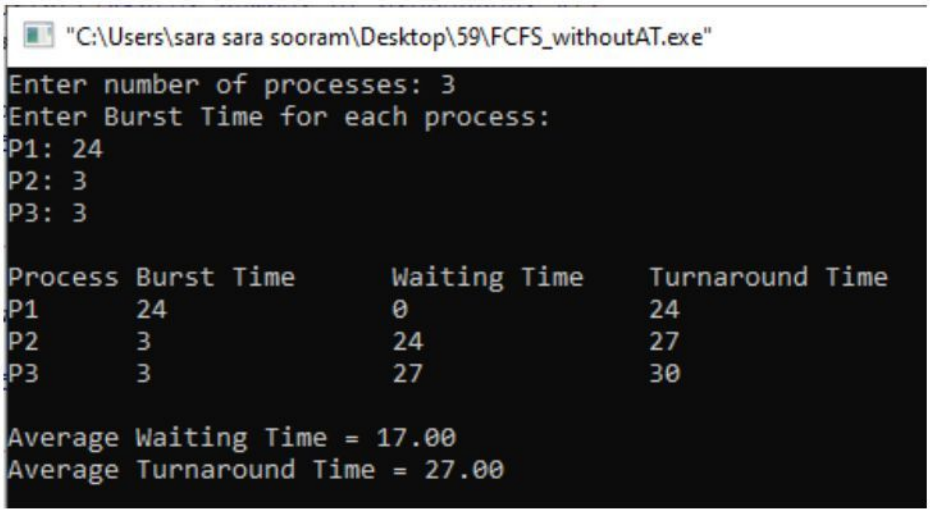
Name of the Laboratory:

Name of the Experiment:

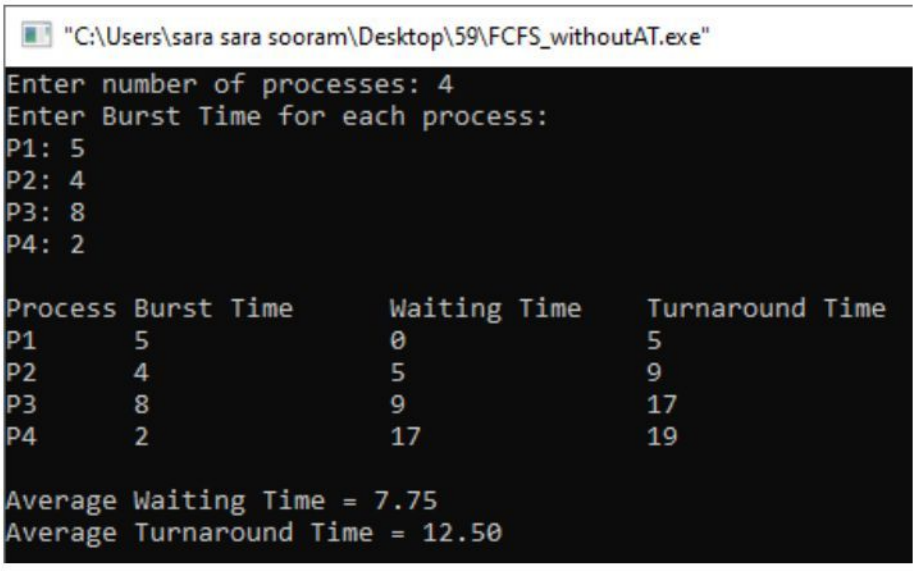
Experiment No: **Date:**

Output:

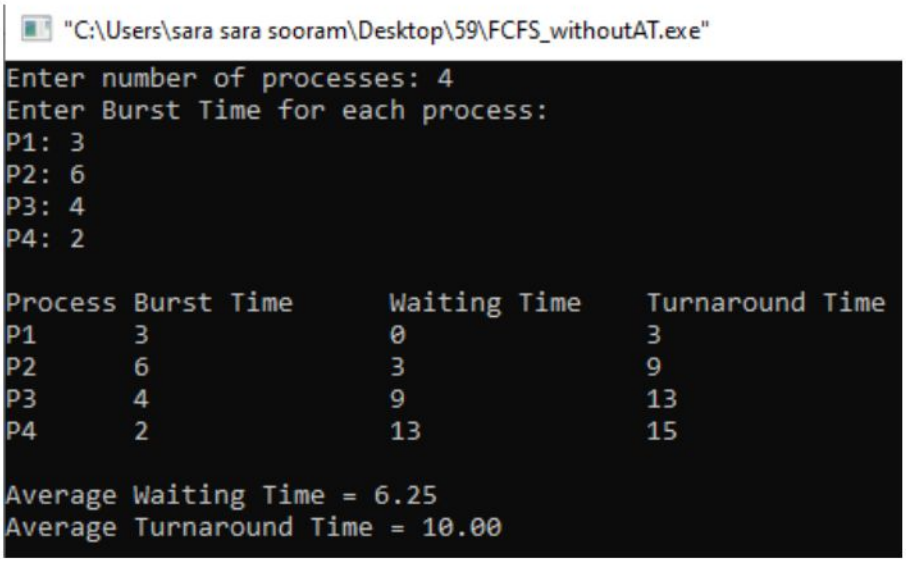
Test Case 1:



Test Case 2:



Test Case 3:



(ii)FCFS with Arrival time

Algorithm:

1. Start.
2. Input the number of processes (n).
3. Input the Burst Time (BT) for each process.
4. Initialize WT[0] = 0, total_wt = 0, and total_tat = 0.
5. For each process:
   - Calculate WT using the previous process's burst time.
   - Calculate TAT = WT + BT.
   - Add WT and TAT to their respective totals.
6. Calculate:
   - Average Waiting Time = Total WT / n
   - Average Turnaround Time = Total TAT / n
7. Display BT, WT, TAT, Average WT, and Average TAT.
8. Stop.

Problem:

Consider the set of processes that arrive at the following times with the burst time given in milliseconds.

Process	Arrival Times	Burst Time
P1	0	5
P2	2	3
P3	4	6

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

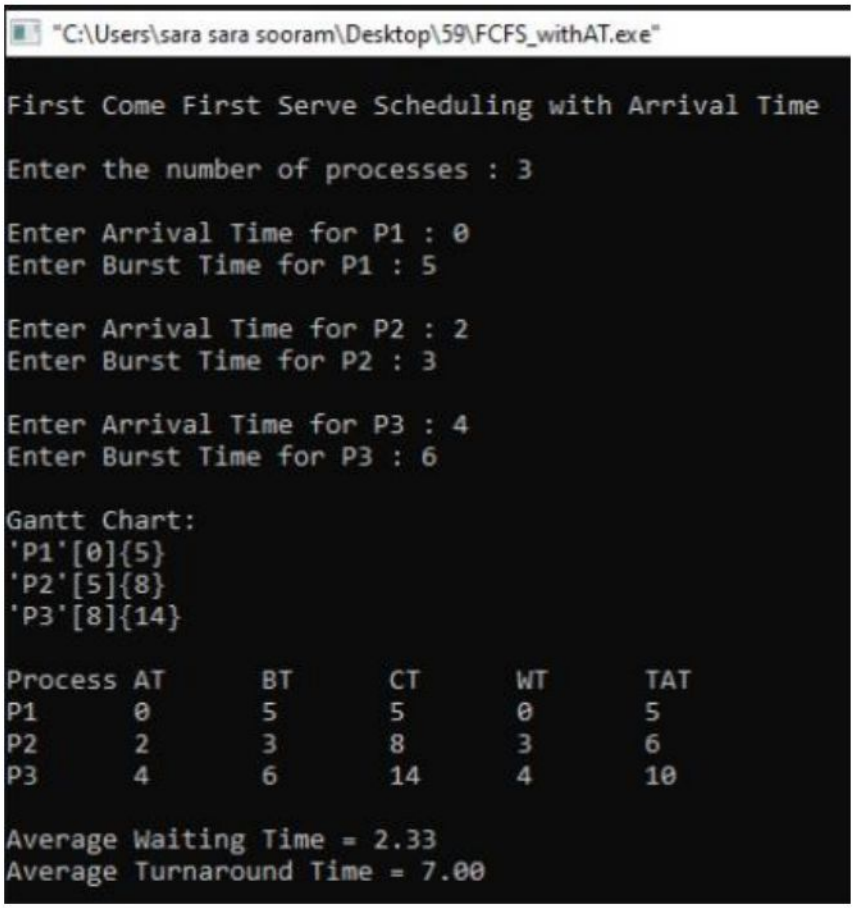
Solution:

Program:

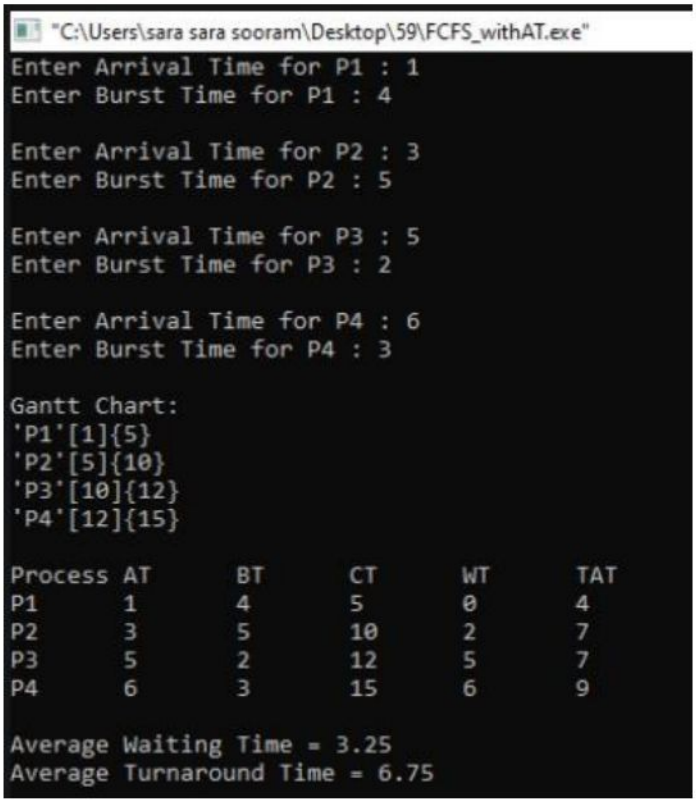
```
#include <stdio.h>
int main() {
int n, at[20], bt[20], time = 0;
float twt = 0, ttat = 0;
printf("Roll no: 25071A6227\n");
printf("Enter number of processes: ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
scanf("%d%d", &at[i], &bt[i]);
}
printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
if (time < at[i]) time = at[i];
time += bt[i];
int tat = time - at[i], wt = tat - bt[i];
twt += wt; ttat += tat;
printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], time, wt, tat);
}
printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
return 0;
}
```

Output:

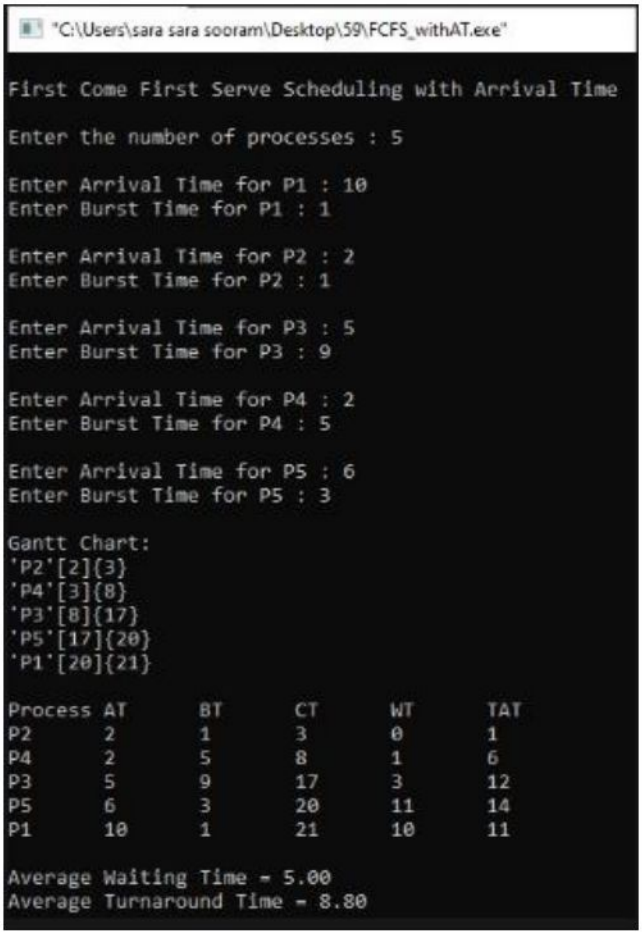
Test Case 1:



Test Case 2:



Test Case 3:.



```
"C:\Users\sara sara sooram\Desktop\59\FCFS_withAT.exe"

First Come First Serve Scheduling with Arrival Time

Enter the number of processes : 5

Enter Arrival Time for P1 : 10
Enter Burst Time for P1 : 1

Enter Arrival Time for P2 : 2
Enter Burst Time for P2 : 1

Enter Arrival Time for P3 : 5
Enter Burst Time for P3 : 9

Enter Arrival Time for P4 : 2
Enter Burst Time for P4 : 5

Enter Arrival Time for P5 : 6
Enter Burst Time for P5 : 3

Gantt Chart:
'P2'[2]{3}
'P4'[3]{8}
'P3'[8]{17}
'P5'[17]{20}
'P1'[20]{21}

Process AT      BT      CT      WT      TAT
P2      2       1       3       0       1
P4      2       5       8       1       6
P3      5       9       17      3       12
P5      6       3       20      11      14
P1      10      1       21      10      11

Average Waiting Time - 5.00
Average Turnaround Time - 8.80
```

(iii) <u>SJF-Shortest Job First:</u>

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the smallest Burst Time (BT).
    - If no process has arrived, increase current_time by 1.
5. Otherwise:
    - Execute the selected process.
    - Calculate CT, TAT, and WT.
    - Update current_time = CT.
    - Mark the process as completed.
6. Calculate the Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
7. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
8. Stop.

Problem:

Consider the set of processes that arrive at the following times with the burst time given in milliseconds.

Process	Arrival Times	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Solution:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Program:

```
#include <stdio.h>
int main() {
   int n, p[10], at[10], bt[10], ct[10], tat[10], wt[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: "); scanf("%d", &n);
for(int i=0; i<n; i++)
{ p[i]=i+1; printf("Enter Arrival Time and Burst Time for P%d: ", i+1); scanf("%d%d", &at[i], &bt[i]); }
   while(done < n) {
      int idx = -1, min_bt = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && bt[i] < min_bt) { min_bt = bt[i]; idx = i; }
      if(idx == -1) cur++;
      else {
         cur += bt[idx]; ct[idx] = cur; tat[idx] = ct[idx] - at[idx];
         wt[idx] = tat[idx] - bt[idx]; twt += wt[idx]; ttat += tat[idx]; vis[idx] = 1; done++;
      }
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;
}
```

Output:

Test case 1:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 2 7
Enter Arrival Time and Burst Time for P2: 4 3
Enter Arrival Time and Burst Time for P3: 5 9

Process  AT       BT       CT       WT       TAT
P1       2        7        9        0        7
P2       4        3        12       5        8
P3       5        9        21       8        17

Average Waiting Time = 4.33
Average Turnaround Time = 10.67
```

Test case 2:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 5 6
Enter Arrival Time and Burst Time for P3: 6 2

Process  AT       BT       CT       WT       TAT
P1       0        3        3        0        3
P2       5        6        11       0        6
P3       6        2        13       5        7

Average Waiting Time = 1.67
Average Turnaround Time = 5.33
```

Test case 3:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 1 4
Enter Arrival Time and Burst Time for P2: 2 8
Enter Arrival Time and Burst Time for P3: 3 5

Process  AT      BT      CT      WT      TAT
P1       1       4       5       0       4
P2       2       8       13      3       11
P3       3       5       18      10      15

Average Waiting Time = 4.33
Average Turnaround Time = 10.00
```

(iii) <u>SRTF-Shortest Remaining Time First:</u>

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Remaining Time (RT)=BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the smallest Remaining Time (RT).
    - If no process is ready, increase time by 1.
    - Otherwise:
        - Execute the selected process for 1 time unit.
        - Decrease RT by 1 and increase time by 1.
        - If RT = 0, calculate CT, mark the process as completed, and increment completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

Problem:

Consider the set of processes that arrive at the following times with the burst time given in milliseconds.

Process	Arrival Times	Burst Time
P1	0	7
P2	1	5
P3	2	3
P4	3	1
P5	4	2
P6	5	1

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

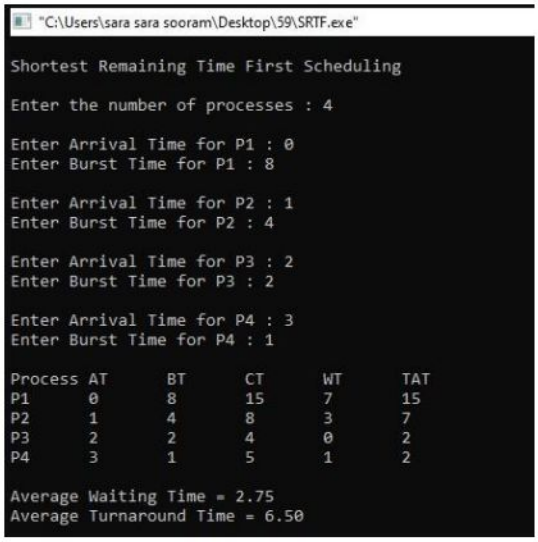
Solution:

Program:

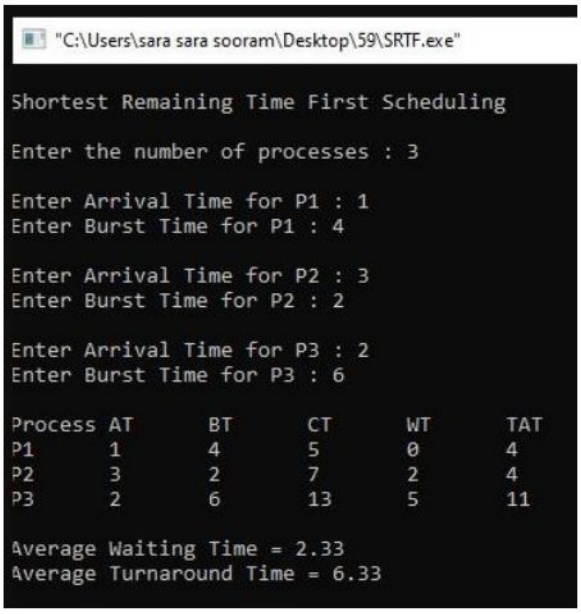
```
#include <stdio.h>
int main() {
    int n, at[10], bt[10], rt[10], ct[10], done = 0, time = 0, min_rt, idx;
    float twt = 0, ttat = 0;
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i]; }
    while (done < n) {
        min_rt = 1e9; idx = -1;
        for (int i = 0; i < n; i++)
            if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt) { min_rt = rt[i]; idx = i; }
        if (idx == -1) { time++; continue; }
        rt[idx]--; time++;
        if (rt[idx] == 0) { done++; ct[idx] = time; } }
    printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        twt += wt; ttat += tat;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat);
    }
    printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
    return 0; }
```

Output:

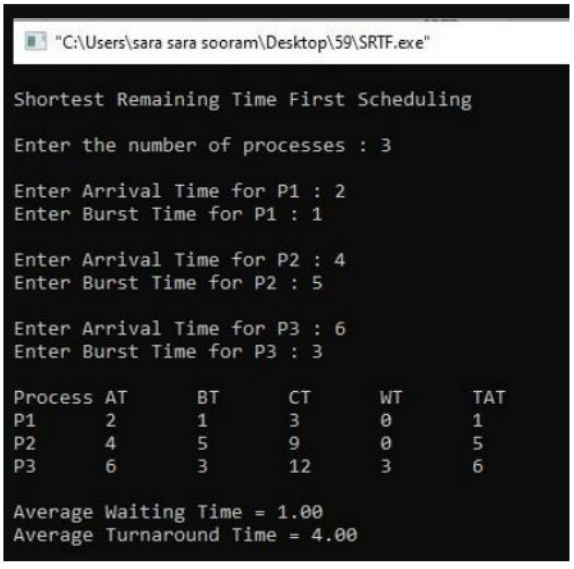
Test case 1:



Test case 2:



Test case 3:



VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Aim: Write a C-program for priority scheduling algorithm A. Non-preemptive B. Preemptive

A. Non-preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the highest priority (lowest Priority value)
    - If no process is ready, increase current_time by 1.
    - Otherwise:
        - Execute the selected process.
        - Calculate CT, TAT, and WT.
        - Update current_time = CT.
        - Mark the process as completed.
5. Calculate Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

Problem:

Consider the set of processes with priorities and burst time given in milliseconds.

Process	BurstTime	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

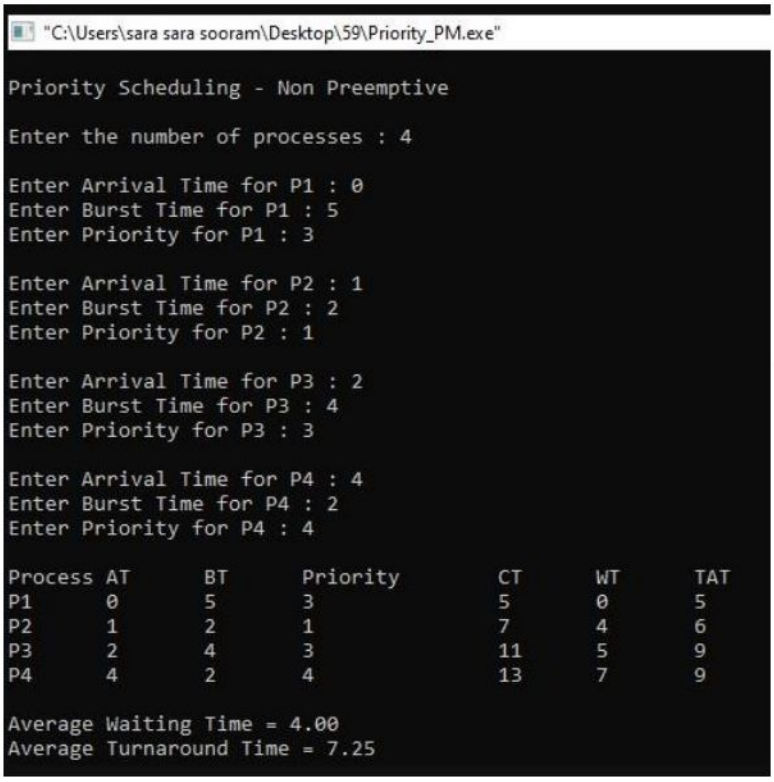
Solution:

Program:

```
#include <stdio.h>
int main() {
   int n, at[10], bt[10], pr[10], ct[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(int i=0; i<n; i++) {
 printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i+1); scanf("%d%d%d", &at[i], &bt[i], &pr[i]); }
   while(done < n) {
      int idx = -1, min_pr = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
      if(idx == -1) cur++;
      else { cur += bt[idx]; ct[idx] = cur; vis[idx] = 1; done++; }
   }
   printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;
}
```

Output:

Test case 1:



```
"C:\Users\sara sara sooram\Desktop\59\Priority_PM.exe"

Priority Scheduling - Non Preemptive

Enter the number of processes : 4

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 5
Enter Priority for P1 : 3

Enter Arrival Time for P2 : 1
Enter Burst Time for P2 : 2
Enter Priority for P2 : 1

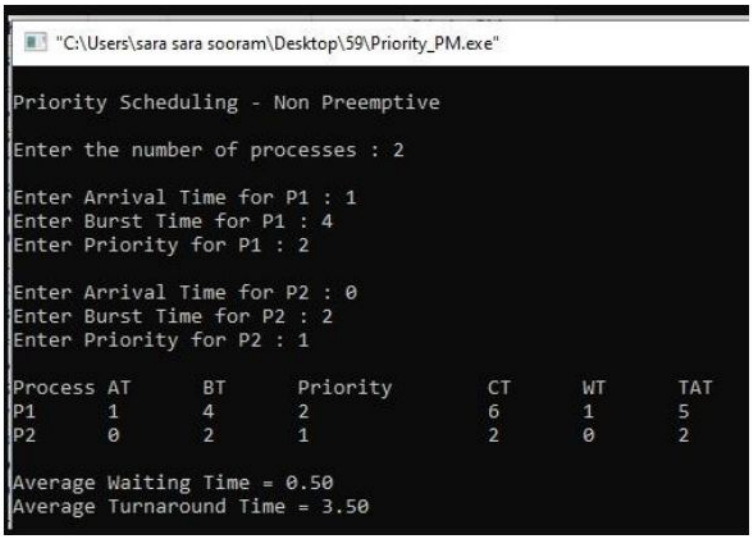
Enter Arrival Time for P3 : 2
Enter Burst Time for P3 : 4
Enter Priority for P3 : 3

Enter Arrival Time for P4 : 4
Enter Burst Time for P4 : 2
Enter Priority for P4 : 4

Process AT      BT      Priority        CT      WT      TAT
P1      0       5       3               5       0       5
P2      1       2       1               7       4       6
P3      2       4       3               11      5       9
P4      4       2       4               13      7       9

Average Waiting Time = 4.00
Average Turnaround Time = 7.25
```

Test case 2:



```
"C:\Users\sara sara sooram\Desktop\59\Priority_PM.exe"

Priority Scheduling - Non Preemptive

Enter the number of processes : 2

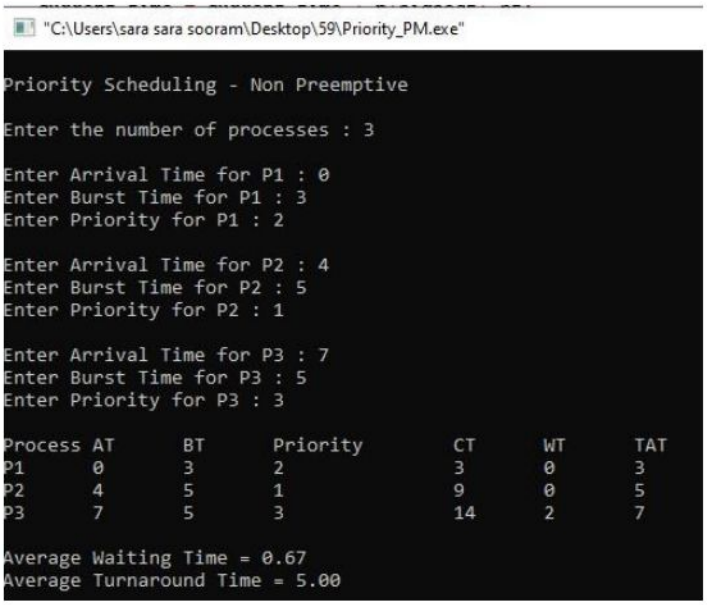
Enter Arrival Time for P1 : 1
Enter Burst Time for P1 : 4
Enter Priority for P1 : 2

Enter Arrival Time for P2 : 0
Enter Burst Time for P2 : 2
Enter Priority for P2 : 1

Process AT      BT      Priority        CT      WT      TAT
P1      1       4       2               6       1       5
P2      0       2       1               2       0       2

Average Waiting Time = 0.50
Average Turnaround Time = 3.50
```

Test case 3:



```
"C:\Users\sara sara sooram\Desktop\59\Priority_PM.exe"

Priority Scheduling - Non Preemptive

Enter the number of processes : 3

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 3
Enter Priority for P1 : 2

Enter Arrival Time for P2 : 4
Enter Burst Time for P2 : 5
Enter Priority for P2 : 1

Enter Arrival Time for P3 : 7
Enter Burst Time for P3 : 5
Enter Priority for P3 : 3

Process AT      BT      Priority        CT      WT      TAT
P1      0       3       2               3       0       3
P2      4       5       1               9       0       5
P3      7       5       3               14      2       7

Average Waiting Time = 0.67
Average Turnaround Time = 5.00
```

B. Preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr). Set Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the highest priority (lowest Priority value).
    - If no process is ready, increase time by 1.

- Otherwise:
    - Execute the selected process for 1 time unit.
    - Decrease RT by 1 and increase time by 1.
    - If RT = 0, calculate CT, mark the process as completed, and increment completed.

5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

Problem:

Consider the set of processes with priorities and burst time given in milliseconds.

Process	BurstTime	Priority
P1	3	2
P2	6	4
P3	4	1
P4	2	3

Solution:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

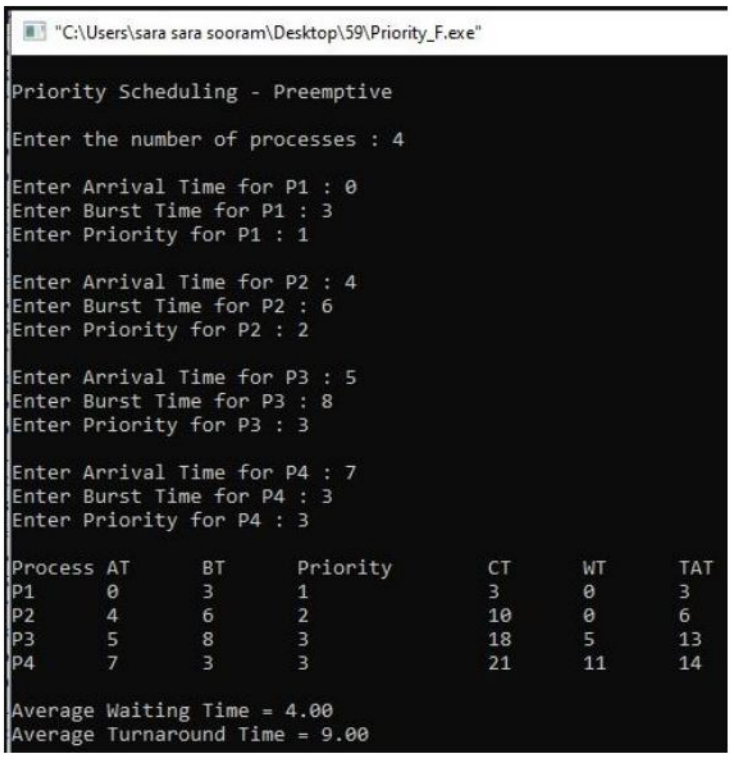
Experiment No: **Date:**

Program:

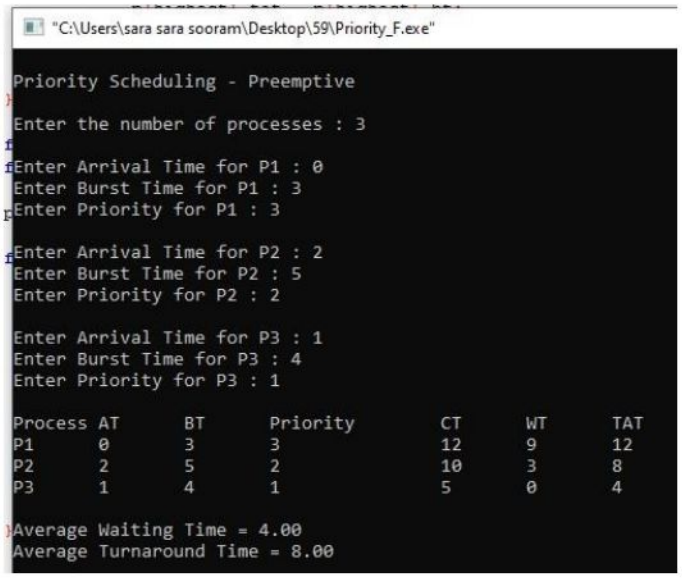
```
#include <stdio.h>
int main() {
    int n, at[10], bt[10], pr[10], rt[10], ct[10], done = 0, time = 0, min_pr, idx;
    float twt = 0, ttat = 0;
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i + 1);
        scanf("%d%d%d", &at[i], &bt[i], &pr[i]); rt[i] = bt[i];
    }
    while (done < n) {
        min_pr = 1e9; idx = -1;
        for (int i = 0; i < n; i++)
            if (at[i] <= time && rt[i] > 0 && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
        if (idx == -1) { time++; continue; }
        rt[idx]--; time++;
        if (rt[idx] == 0) { done++; ct[idx] = time; }
    }
    printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], ct[i], wt, tat);
    }
    printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
    return 0;
}
```

Output:

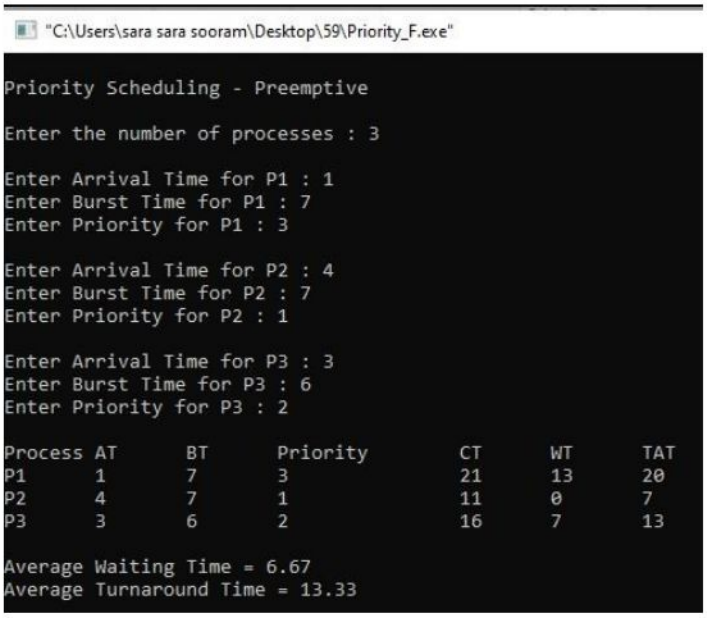
Test case 1:



Test case 2:



Test case 3:



Aim: write a c-program for Round Robin scheduling algorithm A. Time Quantum=2, B. Time Quantum=10

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Time Quantum (TQ) = 2 and Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the first process from the Ready Queue.
    - Execute the process for 2 units or until it finishes.
    - Update RT and time.
    - Add any newly arrived processes to the Ready Queue.
    - If RT > 0, add the process back to the Ready Queue; otherwise, calculate CT and mark it as completed.

1. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
2. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
3. Stop.

Problem:

Consider the set of processes with burst time given in milliseconds and time quantum=2 and 10ms.

Process	BurstTime
P1	4
P2	5
P3	3

Solution:

Program:

```
#include <stdio.h>
int main() {
printf("Roll no: 25071A6227\n");
int n, tq = 2, at[10], bt[10], rt[10], ct[10], done = 0, time = 0;
float twt = 0, ttat = 0;
printf("Enter number of processes: "); scanf("%d", &n);
for (int i = 0; i < n; i++) {
printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];}
while (done < n) {
int exec = 0;
for (int i = 0; i < n; i++) {
if (at[i] <= time && rt[i] > 0) {
exec = 1;
int slice = (rt[i] > tq) ? tq : rt[i];
rt[i] -= slice; time += slice;
if (rt[i] == 0) { done++; ct[i] = time; } }}
if (!exec) time++;}
printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat); }
printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
return 0;}
```

Output: Time Quantum = 2ms

Test case 1:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 5

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 1

Enter Arrival Time for P2 : 2
Enter Burst Time for P2 : 4

Enter Arrival Time for P3 : 6
Enter Burst Time for P3 : 3

Enter Arrival Time for P4 : 5
Enter Burst Time for P4 : 2

Enter Arrival Time for P5 : 4
Enter Burst Time for P5 : 7

Enter Time Quantum : 2

Process AT      BT      CT      WT      TAT
P1      0       1       1       0       1
P2      2       4       8       2       6
P3      6       3       15      6       9
P4      5       2       12      5       7
P5      4       7       18      7       14

Average Waiting Time = 4.00
Average Turnaround Time = 7.40
```

Test case 2:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 6

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 4

Enter Arrival Time for P2 : 1
Enter Burst Time for P2 : 5

Enter Arrival Time for P3 : 2
Enter Burst Time for P3 : 2

Enter Arrival Time for P4 : 3
Enter Burst Time for P4 : 1

Enter Arrival Time for P5 : 4
Enter Burst Time for P5 : 6

Enter Arrival Time for P6 : 5
Enter Burst Time for P6 : 3

Enter Time Quantum : 2

Process AT      BT      CT      WT      TAT
P1      0       4       13      9       13
P2      1       5       19      13      18
P3      2       2       6       2       4
P4      3       1       7       3       4
P5      4       6       21      11      17
P6      5       3       18      10      13

Average Waiting Time = 8.00
Average Turnaround Time = 11.50
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Test case 3:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 3

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 10

Enter Arrival Time for P2 : 0
Enter Burst Time for P2 : 5

Enter Arrival Time for P3 : 0
Enter Burst Time for P3 : 8

Enter Time Quantum : 2

Process AT      BT      CT      WT      TAT
P1      0       10      23      13      23
P2      0       5       15      10      15
P3      0       8       21      13      21

Average Waiting Time = 12.00
Average Turnaround Time = 19.67
```

Time Quantum = 10 Test case 1:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 6

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 4

Enter Arrival Time for P2 : 1
Enter Burst Time for P2 : 5

Enter Arrival Time for P3 : 2
Enter Burst Time for P3 : 2

Enter Arrival Time for P4 : 3
Enter Burst Time for P4 : 1

Enter Arrival Time for P5 : 4
Enter Burst Time for P5 :
6

Enter Arrival Time for P6 : 6
Enter Burst Time for P6 : 3

Enter Time Quantum : 10

Process AT      BT      CT      WT      TAT
P1      0       4       4       0       4
P2      1       5       9       3       8
P3      2       2       11      7       9
P4      3       1       12      8       9
P5      4       6       18      8       14
P6      6       3       21      12      15

Average Waiting Time = 6.33
Average Turnaround Time = 9.83
```

Test case 2:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 3

Enter Arrival Time for P1 : 0
Enter Burst Time for P1 : 10

Enter Arrival Time for P2 : 0
Enter Burst Time for P2 : 5

Enter Arrival Time for P3 : 0
Enter Burst Time for P3 : 8

Enter Time Quantum : 10

Process AT      BT      CT      WT      TAT
P1      0       10      10      0       10
P2      0       5       15      10      15
P3      0       8       23      15      23

Average Waiting Time = 8.33
Average Turnaround Time = 16.00
```

Test case 3:

```
"C:\Users\sara sara sooram\Desktop\59\RoundRobin.exe"

Round Robin Scheduling

Enter the number of processes : 2

Enter Arrival Time for P1 : 4
Enter Burst Time for P1 : 6

Enter Arrival Time for P2 : 7
Enter Burst Time for P2 : 5

Enter Time Quantum : 10

Process AT      BT      CT      WT      TAT
P1      4       6       10      0       6
P2      7       5       15      3       8

Average Waiting Time = 1.50
Average Turnaround Time = 7.00
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment:
Analysis of the scheduling algorithms

Experiment No: ____ **Date.**

Aim: Make the analysis of all scheduling algorithms

Testcases:

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Burst Time for P1: 24
Enter Burst Time for P2: 3
Enter Burst Time for P3: 3

Process  BT       WT       TAT
P1       24       0        24
P2       3        24       27
P3       3        27       30

Avg WT = 17.00 | Avg TAT = 27.00
```

FCFS scheduling without arrival times

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT       BT       CT       WT       TAT
P1      0        24       24       0        24
P2      0        3        27       24       27
P3      0        3        30       27       30

Average Waiting Time = 17.00
Average Turnaround Time = 27.00
```

FCFS scheduling arrival times

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT       BT       CT       WT       TAT
P1      0        24       30       6        30
P2      0        3        3        0        3
P3      0        3        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

SJF scheduling non-preemptive

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT       BT       CT       WT       TAT
P1      0        24       30       6        30
P2      0        3        3        0        3
P3      0        3        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

SJF scheduling preemptive

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT       BT       Pr       CT       WT       TAT
P1      0        24       3        30       6        30
P2      0        3        1        3        0        3
P3      0        3        2        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Priority scheduling non-preemptive

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT       BT       Pr       CT       WT       TAT
P1      0        24       3        30       6        30
P2      0        3        1        3        0        3
P3      0        3        2        6        3        6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Priority scheduling preemptive

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT       BT       CT       WT       TAT
P1      0        24       30       6        30
P2      0        3        9        6        9
P3      0        3        10       7        10

Average Waiting Time = 6.33
Average Turnaround Time = 16.33
```

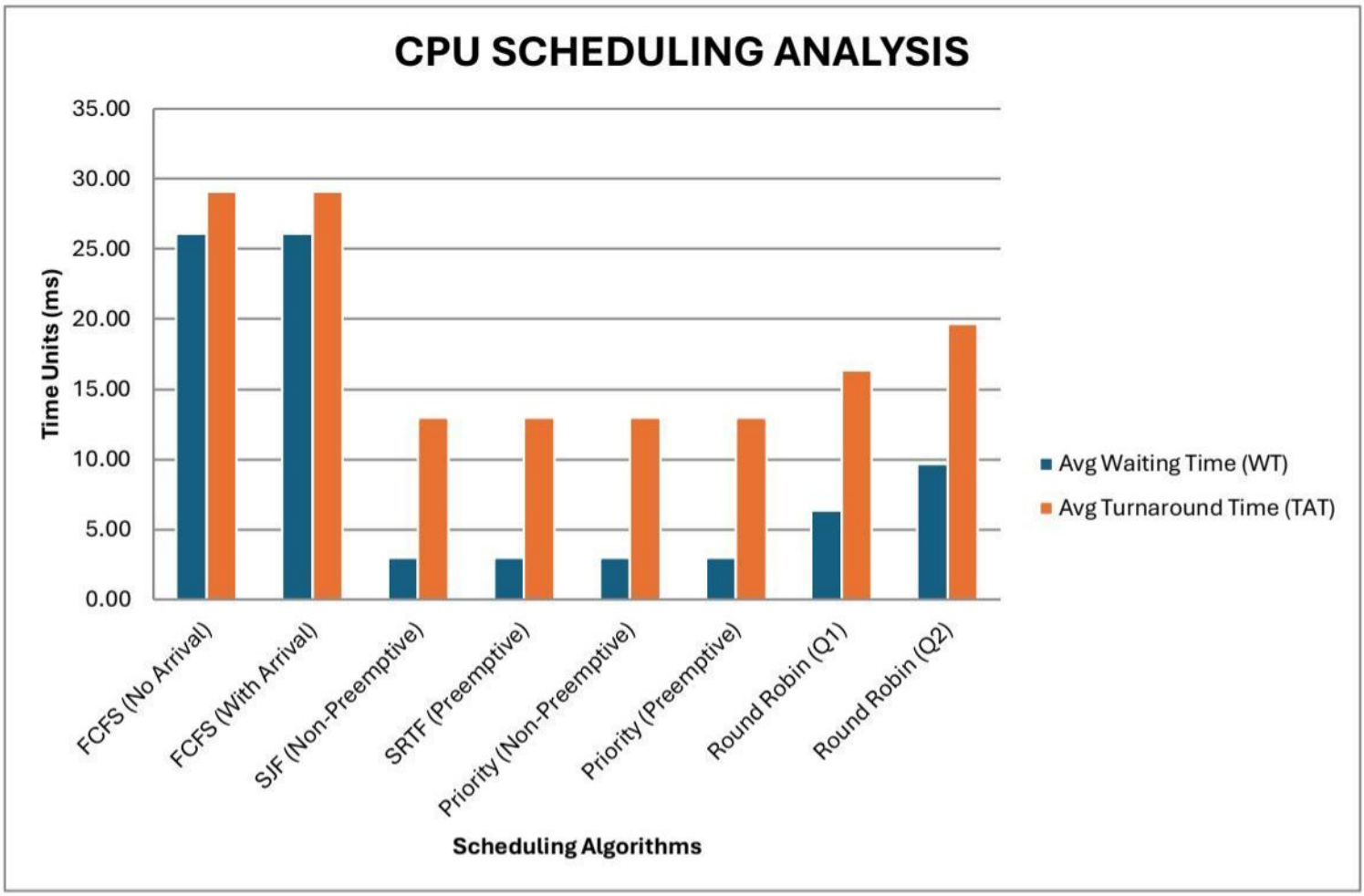
Round Robin scheduling, Time quantum=2

```
Roll no: 25071A6259
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT       BT       CT       WT       TAT
P1      0        24       30       6        30
P2      0        3        13       10       13
P3      0        3        16       13       16

Average Waiting Time = 9.67
Average Turnaround Time = 19.67
```

Round Robin scheduling, Time quantum=10


CPU SCHEDULING ANALYSIS
35.00
30.00
25.00
20.00
15.00
10.00
5.00
0.00
Time Units (ms)
FCFS (No Arrival)
FCFS (With Arrival)
SJF (Non-Preemptive)
SRTF (Preemptive)
Priority (Non-Preemptive)
Priority (Preemptive)
Round Robin (Q1)
Round Robin (Q2)
Scheduling Algorithms
Avg Waiting Time (WT)
Avg Turnaround Time (TAT)

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Simulate Bankers Algorithm for Deadlock Avoidance

Algorithm:

Step 1: Start

Step 2: Read the number of processes (n) and number of resources (m).

Step 3: Read the Allocation matrix.

Step 4: Read the Maximum (Max) matrix.

Step 5: Read the Available resources.

Step 6: Calculate the Need matrix using:
Need[i][j] = Max[i][j] - Allocation[i][j]

Step 7: Initialize:
Work = Available
Finish[i] = false for all processes.

Step 8: Find a process Pi such that:
a) Finish[i] = false
b) Need[i] <= Work

Step 9: If such a process Pi is found, then:
a) Work = Work + Allocation[i]
b) Finish[i] = true
c) Add Pi to the Safe Sequence.
d) Go to Step 8.

Step 10: If all Finish[i] = true, then:
The system is in a SAFE STATE.
Display the Safe Sequence.

Step 11: If all processes cannot be completed, then:
The system is in an UNSAFE STATE.

Step 12: Stop.

Problem: Consider a system with 5 processes and 3 resource types. The complete current state of the system is given below:

Process	Allocation Matrix	Max Matrix	Available Resources
P1	0 1 0	7 5 3	3 3 2
P2	2 0 0	3 2 2	
P3	3 0 2	9 0 2	
P4	2 1 1	2 2 2	
P5	0 2 2	4 3 3	

Program: Source code

```
#include <stdio.h>

 int main() {
 int n, m;

 printf("\nBanker's Algorithm\n");

 printf("\nEnter the number of processes : ");
 scanf("%d", &n);

 printf("Enter the number of resources : ");
 scanf("%d", &m);

 int allocation[n][m];
 int max[n][m];
 int need[n][m];
 int available[m];
 int work[m];
 int finish[n];
 int safeSequence[n];

 printf("\nEnter Allocation Matrix:\n");
 for (int i = 0; i < n; i++) {
    printf("P%d : ", i + 1);
    for (int j = 0; j < m; j++) {
       scanf("%d", &allocation[i][j]);
    }
 }

 printf("\nEnter Max Matrix:\n");
 for (int i = 0; i < n; i++) {
    printf("P%d : ", i + 1);
    for (int j = 0; j < m; j++) {
       scanf("%d", &max[i][j]);
    }
 }

 printf("\nEnter Available Resources:\n");
 for (int j = 0; j < m; j++) {
    scanf("%d", &available[j]);
    work[j] = available[j];
```

```
    for (int i = 0; i < n; i++) {
    finish[i] = 0;

    for (int j = 0; j < m; j++) {
       need[i][j] = max[i][j] - allocation[i][j];
    }
  }
  printf("\nAllocation Matrix:\n");
  printf("\t");
  for (int j = 0; j < m; j++)
    printf("R%d\t", j + 1);

  printf("\n");

  for (int i = 0; i < n; i++) {
    printf("P%d\t", i + 1);

    for (int j = 0; j < m; j++)
       printf("%d\t", allocation[i][j]);

    printf("\n");
  }
  printf("\nMax Matrix:\n");
  printf("\t");
  for (int j = 0; j < m; j++)
    printf("R%d\t", j + 1);

  printf("\n");

  for (int i = 0; i < n; i++) {
    printf("P%d\t", i + 1);

    for (int j = 0; j < m; j++)
       printf("%d\t", max[i][j]);

    printf("\n");
  }
printf("\nNeed Matrix:\n");
  printf("\t");

  for (int j = 0; j < m; j++)
    printf("R%d\t", j + 1);
    printf("\n");
 for (int i = 0; i < n; i++) {
    printf("P%d\t", i + 1);
for (int j = 0; j < m; j++)
       printf("%d\t", need[i][j]);
printf("\n");
  }
```

```
printf("\nInitial Work:\n");
for (int j = 0; j < m; j++)
        printf("%d ", work[j]);
        printf("\n");
        int count = 0;
while (count < n) {
        int found = 0;
  for (int i = 0; i < n; i++) {
     if (finish[i] == 0) {
       int possible = 1;
      for (int j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                   possible = 0;
                   break;
                }
             }
     if (possible) {
                for (int j = 0; j < m; j++) {
                   work[j] = work[j] + allocation[i][j];
                }
safeSequence[count] = i;
                finish[i] = 1;
                count++;
                found = 1;
             printf("\nP%d can execute.", i + 1);
             printf("\nWork after P%d : ", i + 1);
            for (int j = 0; j < m; j++)
                   printf("%d ", work[j]);
                      printf("\n");
             }
          }
       }
```

```
        if (found == 0)
        break;
   }
   if (count == n) {
     printf("\nSystem is in SAFE state.\n");
     printf("\nSafe Sequence: ");
     for (int i = 0; i < n; i++) {
       printf("P%d", safeSequence[i] + 1);
  if (i != n - 1)
            printf(" -> ");
}
 printf("\n");
 }
 else
{
printf("\nSystem is NOT in a safe state.\n");
   } return 0;
}
```

Solution:

Output:

Test case 1:

```
"C:\Users\sara sara sooram\Desktop\59\BankersAlg.exe"

Banker's Algorithm

Enter the number of processes : 5
Enter the number of resources : 3

Enter Allocation Matrix:
P1 : 0 1 0
P2 : 2 0 0
P3 : 3 0 2
P4 : 2 1 1
P5 : 0 2 2

Enter Max Matrix:
P1 : 7 5 3
P2 : 3 2 2
P3 : 9 0 2
P4 : 2 2 2
P5 : 4 3 3

Enter Available Resources:
3 3 2

Allocation Matrix:
        R1      R2      R3
P1      0       1       0
P2      2       0       0
P3      3       0       2
P4      2       1       1
P5      0       2       2

Max Matrix:
        R1      R2      R3
P1      7       5       3
P2      3       2       2
P3      9       0       2
P4      2       2       2
P5      4       3       3
```

```
Enter Available Resources:
3 3 2

Allocation Matrix:
        R1      R2      R3
P1      0       1       0
P2      2       0       0
P3      3       0       2
P4      2       1       1
P5      0       2       2

Max Matrix:
        R1      R2      R3
P1      7       5       3
P2      3       2       2
P3      9       0       2
P4      2       2       2
P5      4       3       3

Need Matrix:
        R1      R2      R3
P1      7       4       3
P2      1       2       2
P3      6       0       0
P4      0       1       1
P5      4       1       1

Initial Work:
3 3 2

P2 can execute.
Work after P2 : 5 3 2

P4 can execute.
Work after P4 : 7 4 3

P5 can execute.
Work after P5 : 7 6 5

P1 can execute.
Work after P1 : 7 7 5
```

```
P2 can execute.
Work after P2 : 5 3 2

P4 can execute.
Work after P4 : 7 4 3

P5 can execute.
Work after P5 : 7 6 5

P1 can execute.
Work after P1 : 7 7 5

P3 can execute.
Work after P3 : 10 7 7

System is in SAFE state.

Safe Sequence: P2 -> P4 -> P5 -> P1 -> P3
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Test case 2 &3:

```
"C:\Users\sara sara sooram\Desktop\59\BankersAlg.exe"

Banker's Algorithm

Enter the number of processes : 2
Enter the number of resources : 3

Enter Allocation Matrix:
P1 : 1 0 1
P2 : 0 2 2

Enter Max Matrix:
P1 : 4 3 2
P2 : 5 1 6

Enter Available Resources:
7 4 5

Allocation Matrix:
        R1      R2      R3
P1      1       0       1
P2      0       2       2

Max Matrix:
        R1      R2      R3
P1      4       3       2
P2      5       1       6

Need Matrix:
        R1      R2      R3
P1      3       3       1
P2      5       -1      4

Initial Work:
7 4 5

P1 can execute.
Work after P1 : 8 4 6

P2 can execute.
Work after P2 : 8 6 8

System is in SAFE state.

Safe Sequence: P1 -> P2
```

```
"C:\Users\sara sara sooram\Desktop\59\BankersAlg.exe"

Banker's Algorithm

Enter the number of processes : 2
Enter the number of resources : 3

Enter Allocation Matrix:
P1 : 1 0 1
P2 : 2 1 1

Enter Max Matrix:
P1 : 5 4 4
P2 : 3 5 9

Enter Available Resources:
9 7 8

Allocation Matrix:
        R1      R2      R3
P1      1       0       1
P2      2       1       1

Max Matrix:
        R1      R2      R3
P1      5       4       4
P2      3       5       9

Need Matrix:
        R1      R2      R3
P1      4       4       3
P2      1       4       8

Initial Work:
9 7 8

P1 can execute.
Work after P1 : 10 7 9

P2 can execute.
Work after P2 : 12 8 10

System is in SAFE state.

Safe Sequence: P1 -> P2
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Implementation of the following Memory Allocation Methods for fixed partition First Fit b) Worst Fit c)Best Fit

Algorithm: First Fit

1. Start.
2. Read the number of memory blocks.
3. Read the size of each memory block.
4. Read the number of processes.
5. Read the size of each process.
6. For each process:
   a. Search all available memory blocks.
   b. Find the smallest block whose size is greater than or equal to the process size.
   c. Allocate the process to that block.
   d. Mark the block as occupied.
   e. If no suitable block is found, mark the process as "Not Allocated".
7. Display the allocation of each process.
8. Stop.

Problem:

Solution:

Program:

```
#include <stdio.h>

int main()
{
   int block[20], process[20];
   int nb, np, i, j;

   printf("Enter number of memory blocks: ");
   scanf("%d", &nb);

   printf("Enter size of memory blocks:\n");
   for(i = 0; i < nb; i++)
      scanf("%d", &block[i]);

   printf("Enter number of processes: ");
   scanf("%d", &np);

   printf("Enter size of processes:\n");
   for(i = 0; i < np; i++)
      scanf("%d", &process[i]);

   printf("\nFirst Fit Allocation:\n");

   for(i = 0; i < np; i++)
   {
      for(j = 0; j < nb; j++)
      {
         if(block[j] >= process[i])
         {
            printf("Process %d -> Block %d\n", i + 1, j + 1);
            block[j] = 0;
            break;
         }
      }

      if(j == nb)
         printf("Process %d -> Not Allocated\n", i + 1);
   }

   return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Output:

Test case 1:

```
Roll no: 25071A6259
Enter number of memory blocks: 5
Enter size of memory blocks:
100
300
200
500
400
Enter number of processes: 3
Enter size of processes:
64
153
89

First Fit Allocation:
Process 1 -> Block 1
Process 2 -> Block 2
Process 3 -> Block 3
```

Test case 2:

```
Roll no: 25071A6259
Enter number of memory blocks: 3
Enter size of memory blocks:
400
200
500
Enter number of processes: 2
Enter size of processes:
500
100

First Fit Allocation:
Process 1 -> Block 3
Process 2 -> Block 1
```

Test case 3:

```
Roll no: 25071A6259
Enter number of memory blocks: 4
Enter size of memory blocks:
250
60
360
180
Enter number of processes: 3
Enter size of processes:
100
89
231

First Fit Allocation:
Process 1 -> Block 1
Process 2 -> Block 3
Process 3 -> Not Allocated
```

Algorithm: Worst Fit

1. Start.
2. Read the number of memory blocks.
3. Read the size of each memory block.
4. Read the number of processes.
5. Read the size of each process.
6. For each process:
   a. Search all available memory blocks.
   b. Find the largest block whose size is greater than or equal to the process size.
   c. Allocate the process to that block.
   d. Mark the block as occupied.
   e. If no suitable block is found, mark the process as "Not Allocated".
7. Display the allocation of each process.
8. Stop.

Problem:

Solution:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Program:

```
#include <stdio.h>

int main()
{
   int block[20], process[20];
   int nb, np, i, j, worst;
   printf("Enter number of memory blocks: ");
   scanf("%d", &nb);

   printf("Enter size of memory blocks:\n");
   for(i = 0; i < nb; i++)
      scanf("%d", &block[i]);

   printf("Enter number of processes: ");
   scanf("%d", &np);

   printf("Enter size of processes:\n");
   for(i = 0; i < np; i++)
      scanf("%d", &process[i]);

   printf("\nWorst Fit Allocation:\n");

   for(i = 0; i < np; i++)
   {
      worst = -1;

      for(j = 0; j < nb; j++)
      {
         if(block[j] >= process[i])
         {
            if(worst == -1 || block[j] > block[worst])
               worst = j;
         }
      }
      if(worst != -1)
      {
         printf("Process %d -> Block %d\n", i + 1, worst + 1);
         block[worst] = 0;
      }
      else
      {
         printf("Process %d -> Not Allocated\n", i + 1);
      }
   }
   return 0;
}
```

Output:

Test case 1:

```
Roll no: 25071A6259
Enter number of memory blocks: 4
Enter size of memory blocks:
200
400
300
50
Enter number of processes: 2
Enter size of processes:
229
336

Worst Fit Allocation:
Process 1 -> Block 2
Process 2 -> Not Allocated
```

Test case 2:

```
Roll no: 25071A6259
Enter number of memory blocks: 3
Enter size of memory blocks:
400
100
200
Enter number of processes: 3
Enter size of processes:
364
86
184

Worst Fit Allocation:
Process 1 -> Block 1
Process 2 -> Block 3
Process 3 -> Not Allocated
```

Test case 3:

```
Roll no: 25071A6259
Enter number of memory blocks: 2
Enter size of memory blocks:
400
500
Enter number of processes: 3
Enter size of processes:
30
70
200

Worst Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 1
Process 3 -> Not Allocated
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Algorithm: Best Fit

1. Start.
2. Read the number of memory blocks.
3. Read the size of each memory block.
4. Read the number of processes.
5. Read the size of each process.
6. For each process:
   a. Search all available memory blocks.
   b. Find the smallest block whose size is greater than or equal to the process size.
   c. Allocate the process to that block.
   d. Mark the block as occupied.
   e. If no suitable block is found, mark the process as "Not Allocated".
7. Display the allocation of each process.
8. Stop.

Problem:

Solution:

Program:

```
#include <stdio.h>

int main()
{
   int block[20], process[20];
   int nb, np, i, j, best;

   printf("Enter number of memory blocks: ");
   scanf("%d", &nb);

   printf("Enter size of memory blocks:\n");
   for(i = 0; i < nb; i++)
      scanf("%d", &block[i]);

   printf("Enter number of processes: ");
   scanf("%d", &np);

   printf("Enter size of processes:\n");
   for(i = 0; i < np; i++)
      scanf("%d", &process[i]);

   printf("\nBest Fit Allocation:\n");

   for(i = 0; i < np; i++)
   {
      best = -1;

      for(j = 0; j < nb; j++)
      {
         if(block[j] >= process[i])
         {
            if(best == -1 || block[j] < block[best])
               best = j;
         }
      }

      if(best != -1)
      {
         printf("Process %d -> Block %d\n", i + 1, best + 1);
         block[best] = 0;
      }
      else
      {
         printf("Process %d -> Not Allocated\n", i + 1);
      }
   }
   return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Output:

Test case 1:

```
Roll no: 25071A6259
Enter number of memory blocks: 4
Enter size of memory blocks:
100
200
50
400
Enter number of processes: 3
Enter size of processes:
224
32
100

Best Fit Allocation:
Process 1 -> Block 4
Process 2 -> Block 3
Process 3 -> Block 1
```

Test case 2:

```
Roll no: 25071A6259
Enter number of memory blocks: 3
Enter size of memory blocks:
300
30
50
Enter number of processes: 3
Enter size of processes:
20
48
163

Best Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 3
Process 3 -> Block 1
```

Test case 3:

```
Roll no: 25071A6259
Enter number of memory blocks: 5
Enter size of memory blocks:
400
100
200
300
40
Enter number of processes: 4
Enter size of processes:
76
259
335
196

Best Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 4
Process 3 -> Block 1
Process 4 -> Block 3
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Implementation of Paging Technique of Memory Management

Algorithm:

1. Start.
2. Read the page size.
3. Read the number of pages.
4. Read the page table, where each page is mapped to a corresponding frame.
5. Read the logical address.
6. Calculate the page number:
   Page Number = Logical Address / Page Size
7. Calculate the offset:
   Offset = Logical Address % Page Size
8. Check whether the page number is within the valid page range.
9. If the page number is valid:
   a. Get the frame number from the page table.
   b. Calculate the physical address:

   Physical Address = (Frame Number × Page Size) + Offset

10. Display the page number, offset, frame number and physical address.
11. If the page number is invalid, display "Invalid Page Number".
12. Stop.

Problem & Solution:

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

Program:

```
#include <stdio.h>
int main()
{
   int page_size, num_pages;
   int page_table[20];
   int logical_address;
   int page_number, offset;
   int frame_number, physical_address;
   int i;

   printf("Enter page size: ");
   scanf("%d", &page_size);

   printf("Enter number of pages: ");
   scanf("%d", &num_pages);

   printf("Enter page table:\n");

   for(i = 0; i < num_pages; i++)
   {
      printf("Page %d -> Frame: ", i);
      scanf("%d", &page_table[i]);
   }
   printf("Enter logical address: ");
   scanf("%d", &logical_address);

   page_number = logical_address / page_size;
   offset = logical_address % page_size;

   printf("\nPage Number = %d", page_number);
   printf("\nOffset = %d", offset);

   if(page_number >= num_pages)
   {
      printf("\nInvalid Page Number\n");
   }
   else
   {
      frame_number = page_table[page_number];

      physical_address = (frame_number * page_size) + offset;

      printf("\nFrame Number = %d", frame_number);
      printf("\nPhysical Address = %d\n", physical_address);
   }
   return 0;
}
```

Output:

Test case:1

```
Roll no: 25071A6259
Enter page size: 100
Enter number of pages: 4
Enter page table:
Page 0 -> Frame: 5
Page 1 -> Frame: 2
Page 2 -> Frame: 8
Page 3 -> Frame: 1
Enter logical address: 250

Page Number = 2
Offset = 50
Frame Number = 8
Physical Address = 850
```

Test case: 2

```
Roll no: 25071A6259
Enter page size: 100
Enter number of pages: 3
Enter page table:
Page 0 -> Frame: 4
Page 1 -> Frame: 6
Page 2 -> Frame: 1
Enter logical address: 180

Page Number = 1
Offset = 80
Frame Number = 6
Physical Address = 680
```

Test case 3:

```
Roll no: 25071A6259
Enter page size: 200
Enter number of pages: 5
Enter page table:
Page 0 -> Frame: 3
Page 1 -> Frame: 5
Page 2 -> Frame: 1
Page 3 -> Frame: 6
Page 4 -> Frame: 9
Enter logical address: 120

Page Number = 0
Offset = 120
Frame Number = 3
Physical Address = 720
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Implementation of the following Page Replacement Algorithms a) FIFO b) LRU c) optimal

Algorithm: FIFO

1. Start.
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of frames.
5. Initially, set all frames as empty.
6. For each page in the reference string:
   a. Check whether the page is already present in a frame.
   b. If present, it is a Page Hit.
   c. If not present, it is a Page Fault.
   d. Replace the oldest page using FIFO.
7. Display the contents of frames after each page.
8. Display the total number of page faults.
9. Stop.

Algorithm: LRU

1. Start.
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of frames.
5. Initially, set all frames as empty.
6. For each page:
   a. Check whether the page is already present.
   b. If present, it is a Page Hit.
   c. If not present, it is a Page Fault.
   d. If an empty frame is available, place the page there.
   e. Otherwise, find the page that has not been used for the longest time.
   f. Replace that page with the new page.
7. Display the contents of frames after each page.
8. Display the total number of page faults.
9. Stop.

Algorithm: Optimal

1. Start.
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of frames.
5. Initially, set all frames as empty.
6. For each page:
   a. Check whether the page is already present.
   b. If present, it is a Page Hit.
   c. If not present, it is a Page Fault.
   d. If an empty frame is available, place the page there.

e. Otherwise, look at the future page references.
f. Find the page in the frame that will be used farthest in the future.
g. Replace that page.

7. Display the contents of frames after each page.
8. Display the total number of page faults.
9. Stop.

Problem & Solutions:

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Program: FIFO

```
#include <stdio.h>
int main()
{
   int pages[50], frames[10];
   int n, f;
   int i, j;
   int pointer = 0;
   int page_faults = 0;
   int found;
   printf("Enter number of pages: ");
   scanf("%d", &n);
   printf("Enter page reference string:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &pages[i]);
   }
   printf("Enter number of frames: ");
   scanf("%d", &f);

   for(i = 0; i < f; i++)
   {
      frames[i] = -1;
   }
   printf("\nFIFO Page Replacement:\n");

   for(i = 0; i < n; i++)
   {
      found = 0;

      for(j = 0; j < f; j++)
      {
         if(frames[j] == pages[i])
         {
            found = 1;
            break;
         }
      }

      if(found == 0)
      {
         frames[pointer] = pages[i];
         pointer = (pointer + 1) % f;
         page_faults++;
      }
      printf("Page %d: ", pages[i]);

for(j = 0; j < f; j++)

if(frames[j] == -1)
   printf("- ");
else
```

```
        printf("%d ", frames[j]);
        }
        if(found == 0)
           printf("Page Fault");
        else
           printf("Page Hit");
        printf("\n");
    }
    printf("\nTotal Page Faults = %d\n", page_faults)
    return 0;
}
```

Program: LRU

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10], recent[10];
    int n, f;
    int I, j;
    int page_faults = 0;
    int found;
    int pos, oldest;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(I = 0; I < n; i++)
    {
       scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &f);
    for(I = 0; I < f; i++)
    {
       frames[i] = -1;
       recent[i] = -1;
    }
    printf("\Nlru Page Replacement:\n");
    for(I = 0; I < n; i++)
{
       found = 0;

for(j = 0; j < f; j++)
{
  if(frames[j] == pages[i])
  {
    found = 1;
    recent[j] = I;
    break;
          }
```

```
    }

    if(found == 0)
    {
        page_faults++;
        pos = -1;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }
        if(pos == -1)
        {
            oldest = recent[0];
            pos = 0;
            for(j = 1; j < f; j++)
            {
                if(recent[j] < oldest)
                {
                    oldest = recent[j];
                    pos = j;
                }
            }
        }
        frames[pos] = pages[i];
        recent[pos] = I;
    }
    printf("Page %d: ", pages[i]);
    for(j = 0; j < f; j++)
    {
        if(frames[j] == -1)
            printf("- ");
        else
            printf("%d ", frames[j]);
    }
    if(found == 0)
        printf("Page Fault");
    else
        printf("Page Hit");

    printf("\n");
  }
  printf("\nTotal Page Faults = %d\n", page_faults);
  return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program: Optimal

```
#include <stdio.h>
int main()
{
   int pages[50], frames[10];
   int n, f;
   int I, j, k;
   int found;
   int page_faults = 0;
   int pos, farthest, future;
   printf("Enter number of pages: ");
   scanf("%d", &n);
   printf("Enter page reference string:\n");
   for(I = 0; I < n; i++)
   {
      scanf("%d", &pages[i]);
   }
   printf("Enter number of frames: ");
   scanf("%d", &f);
   for(I = 0; I < f; i++)
   {
      frames[i] = -1;
   }
   printf("\nOptimal Page Replacement:\n");
   for(I = 0; I < n; i++)
   {
      found = 0;
  for(j = 0; j < f; j++)
      {
         if(frames[j] == pages[i])
         {
            found = 1;
            break;
         }
      }
      if(found == 0)
      {
         page_faults++;
pos = -1;

for(j = 0; j < f; j++)
{
   if(frames[j] == -1)
   {
      pos = j;
      break;
   }
}
if(pos == -1)
```

```
{
  farthest = -1;
          pos = 0;
          for(j = 0; j < f; j++)
          {
             future = -1;
             for(k = I + 1; k < n; k++)
             {
                if(frames[j] == pages[k])
                {
                   future = k;
                   break;
                }
             }
             if(future == -1)
             {
                pos = j;
                break;
             }
             if(future > farthest)
             {
                farthest = future;
                pos = j;
             }
          }
       }
       frames[pos] = pages[i];
    }
    printf("Page %d: ", pages[i]);
    for(j = 0; j < f; j++)
    {
       if(frames[j] == -1)
          printf("- ");
       else
          printf("%d ", frames[j]);
    if(found == 0)
       printf("Page Fault");
    else
       printf("Page Hit");
    printf("\n");
  }
  printf("\nTotal Page Faults = %d\n", page_faults);
  return 0;
}
```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Output: FIFO

Test case: 1

```
Roll no: 25071A6259
Enter number of pages: 9
Enter page reference string:
1 2 3 4 5 3 2 1 4
Enter number of frames: 3

FIFO Page Replacement:
Page 1: 1 - - Page Fault
Page 2: 1 2 - Page Fault
Page 3: 1 2 3 Page Fault
Page 4: 4 2 3 Page Fault
Page 5: 4 5 3 Page Fault
Page 3: 4 5 3 Page Hit
Page 2: 4 5 2 Page Fault
Page 1: 1 5 2 Page Fault
Page 4: 1 4 2 Page Fault

Total Page Faults = 8
```

Test case 2:

```
Roll no: 25071A6259
Enter number of pages: 12
Enter page reference string:
1 4 2 5 9 8 7 3 6 2 3 9
Enter number of frames: 4

FIFO Page Replacement:
Page 1: 1 - - - Page Fault
Page 4: 1 4 - - Page Fault
Page 2: 1 4 2 - Page Fault
Page 5: 1 4 2 5 Page Fault
Page 9: 9 4 2 5 Page Fault
Page 8: 9 8 2 5 Page Fault
Page 7: 9 8 7 5 Page Fault
Page 3: 9 8 7 3 Page Fault
Page 6: 6 8 7 3 Page Fault
Page 2: 6 2 7 3 Page Fault
Page 3: 6 2 7 3 Page Hit
Page 9: 6 2 9 3 Page Fault

Total Page Faults = 11
```

Test case: 3

```
Roll no: 25071A6259
Enter number of pages: 7
Enter page reference string:
9 6 3 7 2 1 4
Enter number of frames: 6

FIFO Page Replacement:
Page 9: 9 - - - - - Page Fault
Page 6: 9 6 - - - - Page Fault
Page 3: 9 6 3 - - - Page Fault
Page 7: 9 6 3 7 - - Page Fault
Page 2: 9 6 3 7 2 - Page Fault
Page 1: 9 6 3 7 2 1 Page Fault
Page 4: 4 6 3 7 2 1 Page Fault

Total Page Faults = 7
```

Output: LRU

Test case: 1

```
Roll no: 25071A6259
Enter number of pages: 6
Enter page reference string:
1 2 3 6 5 4
Enter number of frames: 2

LRU Page Replacement:
Page 1: 1 - Page Fault
Page 2: 1 2 Page Fault
Page 3: 3 2 Page Fault
Page 6: 3 6 Page Fault
Page 5: 5 6 Page Fault
Page 4: 5 4 Page Fault

Total Page Faults = 6
```

Test case: 2

```
Roll no: 25071A6259
Enter number of pages: 7
Enter page reference string:
7 6 5 4 3 2 1
Enter number of frames: 1

LRU Page Replacement:
Page 7: 7 Page Fault
Page 6: 6 Page Fault
Page 5: 5 Page Fault
Page 4: 4 Page Fault
Page 3: 3 Page Fault
Page 2: 2 Page Fault
Page 1: 1 Page Fault

Total Page Faults = 7
```

Test case: 3

```
Roll no: 25071A6259
Enter number of pages: 5
Enter page reference string:
2 4 6 8 10
Enter number of frames: 4

LRU Page Replacement:
Page 2: 2 - - - Page Fault
Page 4: 2 4 - - Page Fault
Page 6: 2 4 6 - Page Fault
Page 8: 2 4 6 8 Page Fault
Page 10: 10 4 6 8 Page Fault

Total Page Faults = 5
```

Output: Optimal

Test case: 1

```
Roll no: 25071A6259
Enter number of pages: 4
Enter page reference string:
20 36 18 45
Enter number of frames: 9

Optimal Page Replacement:
Page 20: 20 - - - - - - - - Page Fault
Page 36: 20 36 - - - - - - - Page Fault
Page 18: 20 36 18 - - - - - - Page Fault
Page 45: 20 36 18 45 - - - - - Page Fault

Total Page Faults = 4
```

Test case: 2

```
Roll no: 25071A6259
Enter number of pages: 5
Enter page reference string:
7 17 24 99 8
Enter number of frames: 3

Optimal Page Replacement:
Page 7: 7 - - Page Fault
Page 17: 7 17 - Page Fault
Page 24: 7 17 24 Page Fault
Page 99: 99 17 24 Page Fault
Page 8: 8 17 24 Page Fault

Total Page Faults = 5
```

Test case: 3

```
Roll no: 25071A6259
Enter number of pages: 3
Enter page reference string:
93 10 18
Enter number of frames: 1

Optimal Page Replacement:
Page 93: 93 Page Fault
Page 10: 10 Page Fault
Page 18: 18 Page Fault

Total Page Faults = 3
```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Aim: Simulate frame allocation Methods.
a) minimum number of frames b) equal allocation c) proportional allocation

Algorithm: Minimum number of frames

1. Start.
2. Read the number of processes.
3. Read the minimum number of frames required by each process.
4. Calculate the total number of frames required by all processes.
5. Read the total number of available frames.
6. Compare the total required frames with the available frames.
7. If total required frames is less than or equal to the available frames:
   a. Allocate the minimum required frames to each process.
   b. Display the allocation.
8. Otherwise:
   Display "Minimum number of frames cannot be allocated".
9. Stop.

Algorithm: Equal allocation

1. Start.
2. Read the number of processes.
3. Read the total number of available frames.
4. Calculate frames per process:

   Frames per process = Available frames / Number of processes

5. Allocate the calculated number of frames to every process.
6. Calculate the remaining frames:

   Remaining frames = Available frames -
   (Frames per process × Number of processes)

7. Display the number of frames allocated to each process.
8. Display the remaining frames.
9. Stop.

Algorithm: Proportional allocation

1. Start.

2. Read the number of processes.

3. Read the size of each process.

4. Calculate the total size of all processes.

5. Read the total number of available frames.

6. For each process, calculate:

   Frames[i] =
   (Process Size[i] / Total Process Size) × Available Frames

7. Allocate the calculated number of frames to each process.

8. Display the frames allocated to each process.

9. Stop.

Problem and Solution:

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program: Minimum number of frames

```
#include <stdio.h>
int main()
{
   int n, i;
   int required[20];
   int total_required = 0;
   int available;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter minimum frames required by each process:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i + 1);
      scanf("%d", &required[i]);

      total_required += required[i];
   }
   printf("Enter total number of available frames: ");
   scanf("%d", &available);
   printf("\nMinimum Frame Allocation:\n");
   if(total_required <= available)
   {
      for(i = 0; i < n; i++)
      {
         printf("Process %d -> %d frames\n",
               i + 1, required[i]);
      printf("Total frames allocated = %d\n", total_required);
      printf("Remaining frames = %d\n",
            available - total_required);
   }
   else
   {
      printf("Minimum number of frames cannot be allocated.\n");
      printf("Required frames = %d\n", total_required);
      printf("Available frames = %d\n", available);
   }

   return 0;
}

}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program: Equal allocation

```
#include <stdio.h>
 int main()
{
   int n;
   int available;
   int each;
   int remaining;
   int i;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter total number of available frames: ");
   scanf("%d", &available);
   each = available / n;
   remaining = available - (each * n);
   printf("\nEqual Frame Allocation:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d -> %d frames\n",
           i + 1, each);
   }
   printf("Remaining frames = %d\n", remaining);
   return 0;
}
```

Program: Proportional allocation

```
#include <stdio.h>

int main()
{
   int n, i;
   int size[20];
   int frames[20];
   int total_size = 0;
   int available;
   int allocated = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter size of each process:\n");

   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i + 1);
      scanf("%d", &size[i]);
```

```
        total_size += size[i];
    }

    printf("Enter total number of available frames: ");
    scanf("%d", &available);

    printf("\nProportional Frame Allocation:\n");

    for(i = 0; i < n; i++)
    {
        frames[i] = (size[i] * available) / total_size;

        allocated += frames[i];

        printf("Process %d -> %d frames\n",
               i + 1, frames[i]);
    }

    printf("Total frames allocated = %d\n", allocated);
    printf("Remaining frames = %d\n",
           available - allocated);

    return 0;
}
```

Output: Minimum number of frames

Test case: 1

```
Roll no: 25071A6259
Enter number of processes: 4
Enter minimum frames required by each process:
Process 1: 4
Process 2: 5
Process 3: 2
Process 4: 3
Enter total number of available frames: 15

Minimum Frame Allocation:
Process 1 -> 4 frames
Process 2 -> 5 frames
Process 3 -> 2 frames
Process 4 -> 3 frames
Total frames allocated = 14
Remaining frames = 1
```

Test Case: 2

```
Roll no: 25071A6259
Enter number of processes: 4
Enter minimum frames required by each process:
Process 1: 6
Process 2: 4
Process 3: 6
Process 4: 2
Enter total number of available frames: 15

Minimum Frame Allocation:
Minimum number of frames cannot be allocated.
Required frames = 18
Available frames = 15
```

Test case: 3

```
Roll no: 25071A6259
Enter number of processes: 5
Enter minimum frames required by each process:
Process 1: 3
Process 2: 6
Process 3: 1
Process 4: 8
Process 5: 5
Enter total number of available frames: 30

Minimum Frame Allocation:
Process 1 -> 3 frames
Process 2 -> 6 frames
Process 3 -> 1 frames
Process 4 -> 8 frames
Process 5 -> 5 frames
Total frames allocated = 23
Remaining frames = 7
```

Output: Equal allocation

Test case: 1

```
Roll no: 25071A6259
Enter number of processes: 4
Enter total number of available frames: 25

Equal Frame Allocation:
Process 1 -> 6 frames
Process 2 -> 6 frames
Process 3 -> 6 frames
Process 4 -> 6 frames
Remaining frames = 1
```

Test case: 2

```
Roll no: 25071A6259
Enter number of processes: 5
Enter total number of available frames: 8

Equal Frame Allocation:
Process 1 -> 1 frames
Process 2 -> 1 frames
Process 3 -> 1 frames
Process 4 -> 1 frames
Process 5 -> 1 frames
Remaining frames = 3
```

Test case: 3

```
Roll no: 25071A6259
Enter number of processes: 3
Enter total number of available frames: 12

Equal Frame Allocation:
Process 1 -> 4 frames
Process 2 -> 4 frames
Process 3 -> 4 frames
Remaining frames = 0
```

Output: Proportional allocation

Test case: 1

```
Roll no: 25071A6259
Enter number of processes: 4
Enter size of each process:
Process 1: 4
Process 2: 6
Process 3: 1
Process 4: 3
Enter total number of available frames: 10

Proportional Frame Allocation:
Process 1 -> 2 frames
Process 2 -> 4 frames
Process 3 -> 0 frames
Process 4 -> 2 frames
Total frames allocated = 8
Remaining frames = 2
```

Test case: 2

```
Roll no: 25071A6259
Enter number of processes: 5
Enter size of each process:
Process 1: 4
Process 2: 10
Process 3: 2
Process 4: 14
Process 5: 9
Enter total number of available frames: 45

Proportional Frame Allocation:
Process 1 -> 4 frames
Process 2 -> 11 frames
Process 3 -> 2 frames
Process 4 -> 16 frames
Process 5 -> 10 frames
Total frames allocated = 43
Remaining frames = 2
```

Test case: 3

```
Roll no: 25071A6259
Enter number of processes: 3
Enter size of each process:
Process 1: 6
Process 2: 3
Process 3: 9
Enter total number of available frames: 18

Proportional Frame Allocation:
Process 1 -> 6 frames
Process 2 -> 3 frames
Process 3 -> 9 frames
Total frames allocated = 18
Remaining frames = 0
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Simulate the following Disk scheduling algorithms. a. FCFS b) SSTF c) SCAN

Algorithm: FCFS

1. Start
2. Read the number of requests n
3. Read the request queue
4. Read the initial head position
5. Set total head movement = 0
6. For each request in the queue:
   a. Calculate movement = |head - request|
   b. Add movement to total head movement
   c. Move head to the current request
7. Display the total head movement
8. Stop

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program: Source code

```
 #include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, head, total = 0;

   printf("Enter number of requests: ");
   scanf("%d", &n);

   int request[n];

   printf("Enter request queue: ");
   for (int i = 0; i < n; i++)
      scanf("%d", &request[i]);

   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("\nFCFS Order: %d", head);

   for (int i = 0; i < n; i++)
   {
      total += abs(head - request[i]);
      head = request[i];

      printf(" -> %d", head);
   }

   printf("\nTotal Head Movement = %d\n", total);

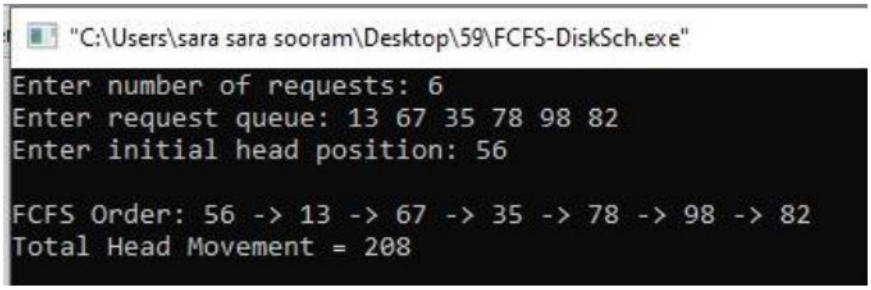
   return 0;
}
```

Output:

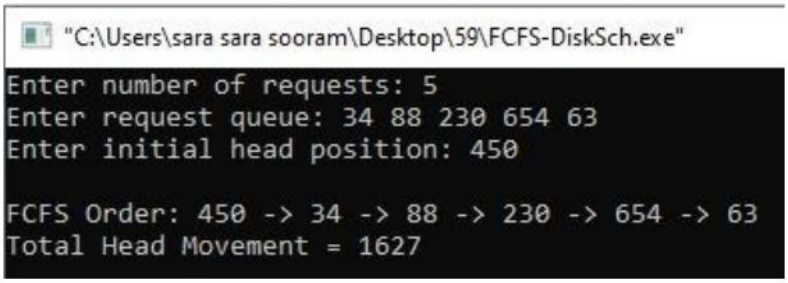
Test case 1:



Test case 2:



Test case 3:



Algorithm: SSTF

1. Start
2. Read the number of requests n
3. Read the request queue
4. Read the initial head position
5. Mark all requests as unvisited
6. Set total head movement = 0
7. Find the unvisited request closest to the current head
8. Calculate movement = |head - request|
9. Add movement to total head movement
10. Mark the selected request as visited
11. Move head to the selected request
12. Repeat steps 7 to 11 until all requests are completed
13. Display the total head movement
14. Stop

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, head, total = 0;

   printf("Enter number of requests: ");
   scanf("%d", &n);
   int request[n], visited[n];

   printf("Enter request queue: ");
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &request[i]);
      visited[i] = 0;
   }
   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("\nSSTF Order: %d", head);

   for (int i = 0; i < n; i++)
   {
      int min = 9999;
      int pos = -1;
      for (int j = 0; j < n; j++)
      {
         if (!visited[j] && abs(head - request[j]) < min)
         {
            min = abs(head - request[j]);
            pos = j;
         }
      }
      visited[pos] = 1;
      total += abs(head - request[pos]);
      head = request[pos];

      printf(" -> %d", head);
   }

   printf("\nTotal Head Movement = %d\n", total);

   return 0;
}
```

VNR VJIET

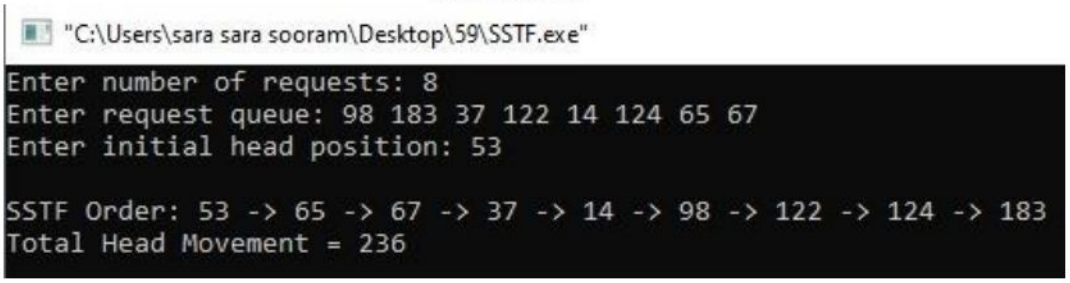
Name of the Laboratory:

Name of the Experiment:_____

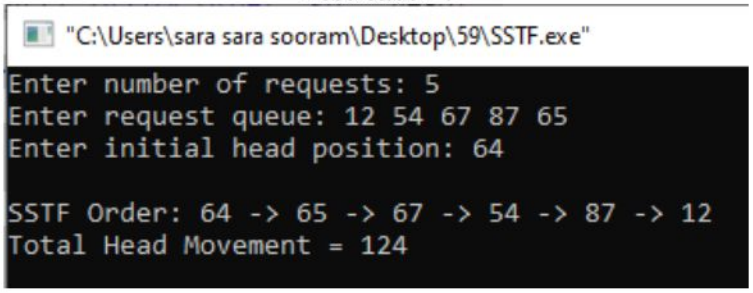
Experiment No:_____ **Date:**_____

Output:

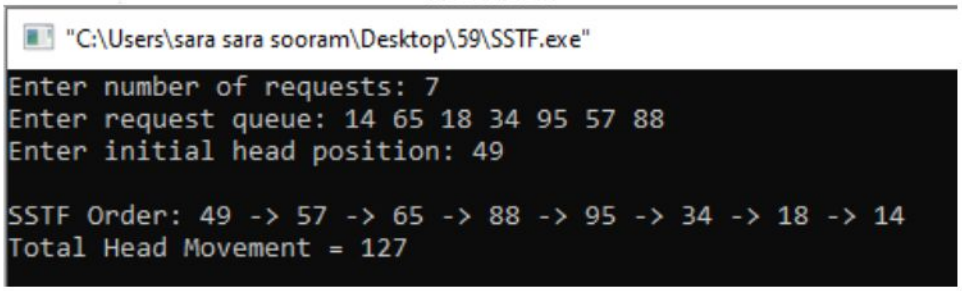
Test case 1:



Test case 2:



Test case 3:



Algorithm: SCAN

1. Start
2. Read the number of requests n
3. Read the request queue
4. Read the initial head position
5. Mark all requests as unvisited
6. Set total head movement = 0
7. Find the unvisited request closest to the current head
8. Calculate movement = |head - request|
9. Add movement to total head movement
10. Mark the selected request as visited
11. Move head to the selected request
12. Repeat steps 7 to 11 until all requests are completed
13. Display the total head movement
14. Stop

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, head, disk_size, total = 0;
   printf("Enter number of requests: ");
   scanf("%d", &n);
   int request[n];
   printf("Enter request queue: ");
   for (int i = 0; i < n; i++)
      scanf("%d", &request[i]);
   printf("Enter initial head position: ");
   scanf("%d", &head);
   printf("Enter disk size: ");
   scanf("%d", &disk_size);
for (int i = 0; i < n - 1; i++)
   {
      for (int j = i + 1; j < n; j++)
      {
         if (request[i] > request[j])
         {
            int temp = request[i];
            request[i] = request[j];
            request[j] = temp;
         }
      }
   }
printf("\nSCAN Order: %d", head);
int pos = 0;
   while (pos < n && request[pos] < head)
      pos++;
```

```
        for (int i = pos; i < n; i++)
    {
        total += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
    total += abs(head - (disk_size - 1));
    head = disk_size - 1;

    printf(" -> %d", head);
    for (int i = pos - 1; i >= 0; i--)
    {
        total += abs(head - request[i]);
        head = request[i];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement = %d\n", total);
    return 0;
}
```

Output:

Test case 1:

```
Enter number of requests: 8
Enter request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

SCAN Order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Head Movement = 331
```

Test case 2:

```
Roll no: 25071A6259
Enter number of requests: 5
Enter request queue: 34 78 81 90 12
Enter initial head position: 43
Enter disk size: 200

SCAN Order: 43 -> 78 -> 81 -> 90 -> 199 -> 34 -> 12
Total Head Movement = 343
```

Test case 3:

```
Roll no: 25071A6259
Enter number of requests: 7
Enter request queue: 12 23 45 36 89 71 57
Enter initial head position: 64
Enter disk size: 200

SCAN Order: 64 -> 71 -> 89 -> 199 -> 57 -> 45 -> 36 -> 23 -> 12
Total Head Movement = 322
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Aim: Simulate the following Disk scheduling algorithms. a. C-SCAN b) LOOK c) C-LOOK

a) C-SCAN

Algorithm:

1. Start.
2. Read the number of disk requests.
3. Read the disk request queue.
4. Read the initial head position.
5. Read the total number of cylinders.
6. Sort the request queue in ascending order.
7. Find the first request greater than or equal to the head position.
8. Move the disk head towards the higher cylinder numbers.
9. Service all requests in this direction.
10. Move the head to the last cylinder.
11. Move the head to the first cylinder without servicing requests during the return movement.
12. Continue moving in the same direction and service the remaining requests.
13. Calculate and display the total head movement.
14. Stop.

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int request[20], n;
   int head, disk_size;
   int i, j, temp;
   int total = 0;

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter the request queue:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &request[i]);

   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("Enter disk size: ");
   scanf("%d", &disk_size);

   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(request[j] > request[j + 1])
         {
            temp = request[j];
            request[j] = request[j + 1];
            request[j + 1] = temp;
         }
      }
   }

   printf("\nC-SCAN Order:\n");

   for(i = 0; i < n; i++)
   {
     if(request[i] >= head)
{
         printf("%d ", request[i]);
```

```
            total += abs(head - request[i]);
            head = request[i];
        }
    }
    total += abs(head - (disk_size - 1));
    head = disk_size - 1;
    total += disk_size - 1;
    head = 0;
    for(i = 0; i < n; i++)
    {
        if(request[i] < head)
        {
            printf("%d ", request[i]);
            total += abs(head - request[i]);
            head = request[i];
        }
    }
    printf("\nTotal Head Movement = %d\n", total);
    return 0;
}
```

Output:

Test case 1:

```
Roll no:25071A6259
Enter number of requests: 8
Enter the request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

C-SCAN Order:
65 67 98 122 124 183 199 0 14 37

Total Head Movement = 382
```

Test case 2:

```
Roll no:25071A6259
Enter number of requests: 5
Enter the request queue:
34 27 98 49 10
Enter initial head position: 67
Enter disk size: 200

C-SCAN Order:
98 199 0 10 27 34 49

Total Head Movement = 380
```

Test case 3:

```
Roll no:25071A6259
Enter number of requests: 6
Enter the request queue:
10 20 30 50 60 70
Enter initial head position: 40
Enter disk size: 200

C-SCAN Order:
50 60 70 199 0 10 20 30

Total Head Movement = 388
```

b) LOOK

Algorithm:

1. Start.
2. Read the number of disk requests.
3. Read the disk request queue.
4. Read the initial head position.
5. Read the direction of head movement.
6. Sort the request queue in ascending order.
7. If the direction is towards higher cylinders:
   a. Service all requests greater than or equal to the head position in ascending order.
   b. Reverse the direction at the last request.
   c. Service the remaining requests in descending order.
8. If the direction is towards lower cylinders:
   a. Service all requests less than or equal to the head position in descending order.
   b. Reverse the direction at the last request.
   c. Service the remaining requests in ascending order.
9. Calculate the total head movement.
10. Display the servicing order and total head movement.
11. Stop.

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int request[20], n;
   int head, direction;
   int i, j, temp;
   int total = 0;

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter the request queue:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &request[i]);

   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("Enter direction (1 for right, 0 for left): ");
   scanf("%d", &direction);
   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(request[j] > request[j + 1])
         {
            temp = request[j];
            request[j] = request[j + 1];
            request[j + 1] = temp;
         }
      }
   }

   printf("\nLOOK Order:\n");

   if(direction == 1)
   {
      for(i = 0; i < n; i++)

if(request[i] >= head)
{
```

```
printf("%d ", request[i]);
total += abs(head - request[i]);
            head = request[i];
         }
      }
      for(i = n - 1; i >= 0; i--)
      {
         if(request[i] < head)
         {
            printf("%d ", request[i]);
            total += abs(head - request[i]);
            head = request[i];
         }
      }
   }
   else
   {
      for(i = n - 1; i >= 0; i--)
      {
         if(request[i] <= head)
         {
            printf("%d ", request[i]);
            total += abs(head - request[i]);
            head = request[i];
         }
      }

      for(i = 0; i < n; i++)
      {
         if(request[i] > head)
         {
            printf("%d ", request[i]);
            total += abs(head - request[i]);
            head = request[i];
         }
      }
   }
  printf("\nTotal Head Movement = %d\n", total);
   return 0;
}
```

Output:

Test case 1:

```
Roll no:25071A6259
Enter number of requests: 8
Enter the request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter direction (1 for right, 0 for left): 0

LOOK Order:
37 14 65 67 98 122 124 183

Total Head Movement = 208
```

```
Roll no:25071A6259
Enter number of requests: 8
Enter the request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter direction (1 for right, 0 for left): 1

LOOK Order:
65 67 98 122 124 183 37 14

Total Head Movement = 299
```

Test case 2:

```
Roll no:25071A6259
Enter number of requests: 4
Enter the request queue:
45 7 18 10
Enter initial head position: 24
Enter direction (1 for right, 0 for left): 0

LOOK Order:
18 10 7 45

Total Head Movement = 55
```

```
Roll no:25071A6259
Enter number of requests: 4
Enter the request queue:
45 7 18 10
Enter initial head position: 24
Enter direction (1 for right, 0 for left): 1

LOOK Order:
45 18 10 7

Total Head Movement = 59
```

Test case 3:

```
Roll no:25071A6259
Enter number of requests: 5
Enter the request queue:
20 10 40 30 70
Enter initial head position: 50
Enter direction (1 for right, 0 for left): 0

LOOK Order:
40 30 20 10 70

Total Head Movement = 100
```

```
Roll no:25071A6259
Enter number of requests: 5
Enter the request queue:
20 10 40 30 70
Enter initial head position: 50
Enter direction (1 for right, 0 for left): 1

LOOK Order:
70 40 30 20 10

Total Head Movement = 80
```

c) C- LOOK

Algorithm:

1. Start.
2. Read the number of disk requests.
3. Read the disk request queue.
4. Read the initial head position.
5. Read the direction of head movement.
6. Sort the request queue in ascending order.
7. If the direction is towards higher cylinders:
   a. Service all requests greater than or equal to the head position in ascending order.
   b. Move directly to the smallest pending request.
   c. Continue servicing the remaining requests in ascending order.
8. If the direction is towards lower cylinders:
   a. Service all requests less than or equal to the head position in descending order.
   b. Move directly to the largest pending request.
   c. Continue servicing the remaining requests in descending order.
9. Calculate the total head movement.
10. Display the servicing order and total head movement.
11. Stop.

Problem: To find the total head movements if queue is 98,183,37,122,14,124,65,67 and head is at 53.

Solution:

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int request[20], n;
   int head, direction;
   int i, j, temp;
   int total = 0;
   int split;

   printf("Enter number of requests: ");
   scanf("%d", &n);

   printf("Enter the request queue:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &request[i]);

   printf("Enter initial head position: ");
   scanf("%d", &head);

   printf("Enter direction (1 for right, 0 for left): ");
   scanf("%d", &direction);

   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(request[j] > request[j + 1])
         {
            temp = request[j];
            request[j] = request[j + 1];
            request[j + 1] = temp;
         }
      }
   }


   split = n;

   for(i = 0; i < n; i++)
   {
```

```
            if(request[i] >= head){
        split = i;
        break;
      }
    }
    printf("\nC-LOOK Order:\n");
    if(direction == 1)
    {
      for(i = split; i < n; i++) {
        printf("%d ", request[i]);
        total += abs(head - request[i]);
        head = request[i];
      }
      for(i = 0; i < split; i++){
        printf("%d ", request[i]);
        total += abs(head - request[i]);
        head = request[i];
      }
    }
else
    {
      for(i = split - 1; i >= 0; i--)
      {
        printf("%d ", request[i]);
        total += abs(head - request[i]);
        head = request[i];
      }

      for(i = n - 1; i >= split; i--)
      {
        printf("%d ", request[i]);
        total += abs(head - request[i]);
        head = request[i];
      }
    } printf("\n\nTotal Head Movement = %d\n", total);
return 0;
    }
```

Output:

Test Case 1:

```
Roll no: 25071A6259
Enter number of requests: 8
Enter the request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter direction (1 for right, 0 for left): 0

C-LOOK Order:
37 14 183 124 122 98 67 65

Total Head Movement = 326
```

```
Roll no: 25071A6259
Enter number of requests: 8
Enter the request queue:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter direction (1 for right, 0 for left): 1

C-LOOK Order:
65 67 98 122 124 183 14 37

Total Head Movement = 322
```

Test Case 2:

```
Roll no: 25071A6259
Enter number of requests: 4
Enter the request queue:
10 40 20 80
Enter initial head position: 60
Enter direction (1 for right, 0 for left): 0

C-LOOK Order:
40 20 10 80

Total Head Movement = 120
```

```
Roll no: 25071A6259
Enter number of requests: 4
Enter the request queue:
10 40 20 80
Enter initial head position: 60
Enter direction (1 for right, 0 for left): 1

C-LOOK Order:
80 10 20 40

Total Head Movement = 120
```

Test Case 3:

```
Roll no: 25071A6259
Enter number of requests: 3
Enter the request queue:
100 150 180
Enter initial head position: 50
Enter direction (1 for right, 0 for left): 0

C-LOOK Order:
180 150 100

Total Head Movement = 210
```

```
Roll no: 25071A6259
Enter number of requests: 3
Enter the request queue:
100 150 180
Enter initial head position: 50
Enter direction (1 for right, 0 for left): 1

C-LOOK Order:
100 150 180

Total Head Movement = 130
```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

Aim: Simulate all file allocation strategies. a. Sequential b) Indexed c) Linked.

Algorithm: Sequential

1. Start.
2. Read the starting block number of the file.
3. Read the number of blocks required by the file.
4. Allocate consecutive blocks starting from the given starting block.
5. Display all the allocated blocks.
6. Stop.

Problem & Solution:

Program:

```
#include <stdio.h>
int main()
{
   int start, length;
   int i;

   printf("Enter starting block: ");
   scanf("%d", &start);

   printf("Enter number of blocks required: ");
   scanf("%d", &length);

   printf("\nSequential File Allocation:\n");
   printf("Allocated Blocks: ");

   for(i = 0; i < length; i++)
   {
      printf("%d ", start + i);
   }

   printf("\n");

   return 0;
}
```

Output:

Test case 1:

```
Roll no: 25071A6259
Enter starting block: 18
Enter number of blocks required: 6

Sequential File Allocation:
Allocated Blocks: 18 19 20 21 22 23
```

Test case 2:

```
Roll no: 25071A6259
Enter starting block: 7
Enter number of blocks required: 3

Sequential File Allocation:
Allocated Blocks: 7 8 9
```

Test case 3:

```
Roll no: 25071A6259
Enter starting block: 45
Enter number of blocks required: 5

Sequential File Allocation:
Allocated Blocks: 45 46 47 48 49
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Algorithm: Indexed File Allocation

1. Start.
2. Read the index block number.
3. Read the number of blocks required by the file.
4. Read the block numbers to be stored in the index block.
5. Store the block numbers in the index table.
6. Display the index block and the allocated data blocks.
7. Stop.

Problem & Solution:

Program:

```
#include <stdio.h>
int main()
{
   int index_block;
   int blocks[20];
   int n;
   int i;
   printf("Enter index block: ");
   scanf("%d", &index_block);
   printf("Enter number of blocks required: ");
```

```
    scanf("%d", &n);
    printf("Enter block numbers:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &blocks[i]);
    }
    printf("\nIndexed File Allocation:\n");
    printf("Index Block: %d\n", index_block);
    printf("Allocated Blocks: ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", blocks[i]);
    }
    printf("\n");
    return 0;
}
```

Output:

Test case 1:

```
Roll no: 25071A6259
Enter index block: 3
Enter number of blocks required: 5
Enter block numbers:
10 93 7 18 45

Indexed File Allocation:
Index Block: 3
Allocated Blocks: 10 93 7 18 45
```

Test case 2:

```
Roll no: 25071A6259
Enter index block: 5
Enter number of blocks required: 4
Enter block numbers:
25 76 23 41

Indexed File Allocation:
Index Block: 5
Allocated Blocks: 25 76 23 41
```

Test case: 3

```
Roll no: 25071A6259
Enter index block: 5
Enter number of blocks required: 3
Enter block numbers:
3 17 18

Indexed File Allocation:
Index Block: 5
Allocated Blocks: 3 17 18
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Algorithm: Linked File Allocation

1. Start.
2. Read the number of blocks required by the file.
3. Read the block numbers in the order in which they are allocated.
4. Link each block to the next block.
5. Set the pointer of the last block to -1.
6. Display the linked list of blocks.
7. Stop.

Problem & Solution:

Program:

```
#include <stdio.h>

int main()
{
   int blocks[20];
   int n;
   int i;
   printf("Enter number of blocks required: ");
   scanf("%d", &n);
   printf("Enter block numbers in allocation order:\n");
```

```
    for(i = 0; i < n; i++)
    {
        scanf("%d", &blocks[i]);
    }
    printf("\nLinked File Allocation:\n");

    for(i = 0; i < n; i++)
    {
        if(i == n - 1)
        {
            printf("Block %d -> NULL\n", blocks[i]);
        }
        else
        {
            printf("Block %d -> Block %d\n", blocks[i], blocks[i + 1]);
        }
    }
    printf("\nFile Allocation Chain:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d", blocks[i]);

        if(i != n - 1)
            printf(" -> ");
    }
    printf(" -> NULL\n");

    return 0;
}
```

Output:

Test case 1:

```
Roll no: 25071A6259
Enter number of blocks required: 5
Enter block numbers in allocation order:
18 7 93 10 45

Linked File Allocation:
Block 18 -> Block 7
Block 7 -> Block 93
Block 93 -> Block 10
Block 10 -> Block 45
Block 45 -> NULL

File Allocation Chain:
18 -> 7 -> 93 -> 10 -> 45 -> NULL
```

Test case: 2

```
Roll no: 25071A6259
Enter number of blocks required: 3
Enter block numbers in allocation order:
14 67 32

Linked File Allocation:
Block 14 -> Block 67
Block 67 -> Block 32
Block 32 -> NULL

File Allocation Chain:
14 -> 67 -> 32 -> NULL
```

Test case: 3

```
Roll no: 25071A6259
Enter number of blocks required: 4
Enter block numbers in allocation order:
90 89 64 33

Linked File Allocation:
Block 90 -> Block 89
Block 89 -> Block 64
Block 64 -> Block 33
Block 33 -> NULL

File Allocation Chain:
90 -> 89 -> 64 -> 33 -> NULL
```